TRIPSSOURCE DESIGN KIT

Implementing the TripsSource design kit

How to use `TripsSource-Design-Kit.pdf` and the files beside it to give another product (DCLab R&D, the DCLab product, or anything new) the TripsSource look, completely and exactly. Follow the steps in order. Each one names the PDF chapter that holds its measurements.

Contents

0. What you have

1. How to read the PDF

2. Install the kit

3. Build the frame (PDF chapter 6)

4. Build every page from a pattern (PDF chapter 8)

5. Build the components (PDF chapter 7)

6. Check your result against the PDF

7. Moving an existing product onto the kit

8. Notes for the DCLab product (`decision_ai/apps/web`)

9. Notes for DCLab R&D

10. A prompt for a coding agent

11. Mistakes to avoid

12. Keeping the kit in step

0. What you have

FILE	WHAT IT IS	WHEN YOU USE IT
`TripsSource-Design-Kit.pdf`	The reference: every token and component, drawn with the kit's own CSS, with values and rules (31 pages).	To know what a thing must look like and measure. Keep it open while you build.
`ts-tokens.css`	Every design value as a `--ts-*` custom property, light and dark, plus the fonts.	Always. It is the single source of every colour, size and radius.
`ts-theme-blue.css`	Blue as the main colour and light green as the accent, in light and dark.	When the product should be blue and light green (DCLab). Leave it out for TripsSource green.
`ts-components.css`	55 components and page layouts as `.ts-*` classes (about 880 rules), plus the app frame.	Always: every component in the PDF comes from here.
`example.html`	Every component on one live page, with palette and light/dark switches.	To copy working markup, and to compare your result with the original in a browser.
`README.md`	The same spec as text.	To search for a value, or to give to a coding agent.
`IMPLEMENTING.md`	This guide.	Now.

The rule that makes it work: the PDF is the target, the CSS files are the implementation. Never retype a value from the PDF into your own CSS. Load the kit's files and use its classes or tokens. A value retyped by hand drifts; a token never does.

1. How to read the PDF

CHAPTER	USE IT FOR
1 Principles	The eight decisions behind the look. Read once before starting; check every new page against them.
2 Setup	The files, fonts, theme attribute and naming rules (step 2 below).
3 Colour	Which token to use for what. Each token has four values: TripsSource light and dark, blue light and dark.
4 Typography	Which size, weight and font each text role uses.
5 Shape and space	Radii, spacing, control heights, shadows, layers, focus.
6 The frame	The sidebar, top bar and canvas (step 3 below).
7 Components	One section per component (step 5 below).
8 Page patterns	The five layouts every page is built from (step 4 below).
9 States	Hover, focus, current, disabled, invalid, loading, motion, hit areas, contrast.
10 Themes	The same components in green, blue light and blue dark. Use it to check your theme.
11 Code	React / Next.js setup, Tailwind, icons.

Each component section in chapter 7 has four parts:

1. **The drawing.** What it must look like. It is drawn by `ts-components.css`, so if yours matches the drawing you have used the kit correctly.
2. **Part / Class.** The class of every part. Build your markup with exactly these classes.
3. **Property / Value.** The measurements and states. Use them to check your result, not to write CSS.
4. **Notes.** When to use the component, and when not to.

2. Install the kit

2.1 Copy the files

Copy the whole `design/tripssource-kit/` folder into the target project, for example to `design/tripssource-kit/` or `app/styles/kit/`. Keep the files unchanged, so the kit can be updated by copying it again. Anything specific to the product goes in a separate file loaded after the kit (see 2.6).

2.2 Load the fonts

The look needs **Outfit** (titles and numbers) and **Plus Jakarta Sans** (everything else), weights 400, 500, 600 and 700.

- **May the product call Google?** Link the fonts:

```
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link rel="stylesheet" href="https://fonts.googleapis.com/css2?
family=Outfit:wght@400;500;600;700&family=Plus+Jakarta+Sans:wght@400;500;600;700&display=swap">
```

- **Must it work offline, or never call Google?** This is the case for the DCLab product (its founder decision D1). Bundle the files instead:
  1. Download the variable woff2 files of Outfit and Plus Jakarta Sans. They are free under the SIL Open Font License; take them from Google Fonts or the `@fontsource-variable/outfit` and `@fontsource-variable/plus-jakarta-sans` packages.
  2. Commit them with their `OFL.txt`, for example under `app/fonts/`.
  3. Load them with `next/font/local`, as in 2.3, and point the kit's font tokens at them.

2.3 Load the CSS (Next.js App Router)

```
// app/layout.tsx
import localFont from 'next/font/local';                  // or: import { Outfit, Plus_Jakarta_Sans } from
'next/font/google'
import '@/design/tripssource-kit/ts-tokens.css';
import '@/design/tripssource-kit/ts-theme-blue.css';      // blue + light green; omit for TripsSource green
import '@/design/tripssource-kit/ts-components.css';
import './globals.css';                                   // your own CSS, after the kit

const display = localFont({ src: './fonts/Outfit-latin-var.woff2', weight: '400 700', variable: '--ts-font-display',
display: 'swap' });
const body = localFont({ src: './fonts/PlusJakartaSans-latin-var.woff2', weight: '400 700', variable: '--ts-font-
body', display: 'swap' });

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en" data-ts-reset className={`${display.variable} ${body.variable}`} suppressHydrationWarning>
      <body>{children}</body>
    </html>
  );
}
```

- **The font variables.** `next/font` sets `--ts-font-display` and `--ts-font-body` on `<html>`, which overrides the kit's defaults, so every component picks up the bundled fonts.
- `data-ts-reset`. It turns on the kit's base: body text, h1 to h6 in Outfit, selection colour, focus ring, thin scrollbars and the type utilities. Add it unless an existing base would fight it; see 7.2 for that case.
- **Plain HTML pages** (for example DCLab R&D's `docs/product-demo/`): use three `<link rel="stylesheet">` tags in the same order, plus the font link from 2.2.

2.4 Light, dark and system

ATTRIBUTE ON `<HTML>`	RESULT
none	Follows the system setting

ATTRIBUTE ON `<HTML>`	RESULT
`data-ts-theme="light"`	Always light
`data-ts-theme="dark"`	Always dark

Offer the choice with the colour-mode component (PDF chapter 7, segmented control). Store it, and set the attribute before the first paint, so the page doesn't flash:

```
<script>
  try { const t = localStorage.getItem('ts-theme'); if (t) document.documentElement.dataset.tsTheme = t; } catch {}
</script>
```

In Next.js put that script in `<head>` with `dangerouslySetInnerHTML`, and keep `suppressHydrationWarning` on `<html>`.

2.5 Tailwind (optional)

Keep the kit's classes for components. Map the tokens only for one-off layout, so a Tailwind class can never invent a colour:

```
// tailwind.config.ts → theme.extend
colors: {
  ground: 'var(--ts-ground)', surface: 'var(--ts-surface)', sunken: 'var(--ts-surface-sunken)',
  ink: 'var(--ts-ink)', text: 'var(--ts-text)', 'text-2': 'var(--ts-text-secondary)', muted: 'var(--ts-text-muted)',
  line: 'var(--ts-border)', 'line-strong': 'var(--ts-border-strong)',
  brand: 'var(--ts-brand)', 'brand-deep': 'var(--ts-brand-deep)', accent: 'var(--ts-accent)', 'accent-soft': 'var(--
ts-accent-soft)',
},
borderRadius: { card: 'var(--ts-radius-card)', pill: 'var(--ts-radius-pill)', sm: 'var(--ts-radius-sm)' },
fontFamily: { display: 'var(--ts-font-display)', body: 'var(--ts-font-body)' },
spacing: { 1: 'var(--ts-space-1)', 2: 'var(--ts-space-2)', 3: 'var(--ts-space-3)', 4: 'var(--ts-space-4)', 5: 'var(-
-ts-space-5)', 6: 'var(--ts-space-6)', 7: 'var(--ts-space-7)', 8: 'var(--ts-space-8)', 9: 'var(--ts-space-9)' },
```

Don't use raw Tailwind colours (`bg-blue-600` , `text-gray-500`) in product screens. Each one is a value the theme cannot reach.

2.6 Your own CSS

Put product-specific rules in one file loaded after the kit, for example `app/styles/product.css`. In it:

- **Use only `var(--ts-*)` .** No hex values, no pixel sizes that aren't in the scale.
- **Give your own classes a product prefix** (for example `.dc-`), never `.ts-` , so the kit can be updated without clashes.
- **To change a token for the product** (for example a slightly larger text size), override it once on `:root, [data-ts-theme]` , and in both dark blocks if it is a colour.

3. Build the frame (PDF chapter 6)

Every signed-in page sits in the same frame. Build it once, as a layout component.

```
// components/AppFrame.tsx
'use client';
import { useEffect, useState } from 'react';

export function AppFrame({ sidebar, topbar, children }: { sidebar: React.ReactNode; topbar: React.ReactNode;
children: React.ReactNode }) {
  const [open, setOpen] = useState(false);
  useEffect(() => {
    const esc = (e: KeyboardEvent) => e.key === 'Escape' && setOpen(false);
    addEventListener('keydown', esc);
    return () => removeEventListener('keydown', esc);
  }, []);
  return (
    <div className="ts-app" data-drawer={open ? 'open' : 'closed'}>
      <aside className="ts-app__sidebar" aria-label="Sidebar">{sidebar}</aside>
      <div className="ts-app__scrim" onClick={() => setOpen(false)} />
      <div className="ts-app__main">
        <header className="ts-topbar">
          <button className="ts-icon-button ts-topbar__menu" data-size="lg" aria-label="Open the menu" onClick={()
=> setOpen(true)}>≡</button>
          {topbar}
        </header>
        <main id="main" className="ts-app__canvas">{children}</main>
      </div>
    </div>
  );
}
```

The sidebar (PDF 7, "Sidebar"). Drive it from one list of sections, so the menu has a single source:

```
type Leaf = { id: string; label: string; href: string };
type Item = { id: string; label: string; icon: React.ReactNode; href?: string; children?: (Leaf | { heading: string
})[] };
type Section = { label: string; items: Item[] };   // e.g. MENU, PROJECTS, THE R&D PRODUCT, GENERAL
```

Render the list with these classes (the full markup is in `example.html`):

- **The brand:** `.ts-sidebar__brand` holds a 38px mark and the name in `.ts-sidebar__brand-name`.
- **Each section:** `.ts-sidebar__nav-section`, with `.ts-sidebar__nav-group-label` for its caps label.
- **A row that is a page:** `<a class="ts-sidebar__nav-row">`. On the current page add `ts-sidebar__nav-row-active` and `aria-current="page"`.
- **A row that is a group:** `<button class="ts-sidebar__nav-row ts-sidebar__nav-row-group" aria-expanded>`, followed by `<ul class="ts-sidebar__nav-children">`.
  - Child rows are `ts-sidebar__nav-row ts-sidebar__nav-row-child`, plus `ts-sidebar__nav-row-current` on the current page.
  - Open the group that holds the current page by itself, and remember which groups were opened.

The top bar. From left to right:

- `.ts-topbar__search`: a pill button that opens your command palette, with `<kbd class="ts-kbd">⌘K</kbd>`;
- then `.ts-topbar__end`, holding round `.ts-icon-button[data-size=lg]` buttons and the `.ts-user-chip`.

The page canvas holds one page: the page header, then sections 22px apart. On a screen narrower than 64rem the sidebar becomes the drawer by itself; your code only toggles `data-drawer`.

4. Build every page from a pattern (PDF chapter 8)

PAGE KIND	BUILD IT AS
Overview / dashboard	Page header → `.ts-stat-grid` with four stat tiles → `.ts-attention-grid` (only cards with something in them) → `.ts-view__layout` with a main panel and a rail
List	Page header with one primary "New" → tabs for saved views → filter bar → data table → table footer; the bulk bar appears when rows are selected
Detail	Page header with breadcrumbs → underline tabs → facts → view cards; an aside from 860px
Form	Page header → form page sections with a sticky section nav → save bar when something changed
Empty or error	An empty state inside the panel where the content would be

Checklist for every page:

- [ ] **One page header:** an Outfit title, one line of subtitle, at most one primary action.
- [ ] **Containers:** every block is a kit component (panel, view card, stat tile, table), never a hand-made box.
- [ ] **Spacing:** 22px between sections, 14px from a panel's header to its body, 12px between tiles.
- [ ] **Counts and numbers:** tabular figures (`.ts-numeric`), right-aligned where they are compared.
- [ ] **Status:** a badge in a tone (success, info, warning, danger, neutral, violet), never a coloured word.
- [ ] **Empty, loading and error:** each has its own state (empty state, skeleton or ring, danger empty state).
- [ ] **Breakpoints:** the page works at 375px, 768px, 1280px and 1600px, with nothing scrolling sideways.

5. Build the components (PDF chapter 7)

Wrap each component once, as a small React component that maps props to the kit's attributes. Pages then use the wrappers and never write `ts-` classes themselves, so a fix is made in one place.

5.1 The props-to-attributes map

COMPONENT	PROPS → ATTRIBUTES	EXAMPLE
Button	`variant` → `data-variant` (primary, accent, secondary, danger, ghost); `size` → `data-size` (sm, md, lg); `loading` → `data-state="loading"`	`<button class="ts-button" data-variant="primary" data-size="md">`
IconButton	`size` → `data-size`; `label` → `aria-label` (required)	`<button class="ts-icon-button" data-size="lg" aria-label="Alerts">`
Badge	`tone` → `data-tone`; `caps` → `data-caps`; `solid` → `data-solid`	`<span class="ts-badge" data-tone="warning" data-caps>`
Chip	`onRemove` → `.ts-chip__remove-button`	`<span class="ts-chip"><span class="ts-chip__label">…`
StatTile	`tone` → `__icon-chip[data-icon-tone]`; `trend` → `__change[data-tone=positive|negative]`	see 5.2
Panel	`title`, `subtitle`, `actions` → `__header`, `__title`, `__subtitle`, `__actions`	`<section class="ts-panel">`
PageHeader	`crumbs`, `title`, `subtitle`, `actions`	`<header class="ts-page-header">`
Tabs	`variant` → `__list[data-variant=underline|pill]`; selected → `aria-selected`	`<div class="ts-tabs__list" data-variant="underline" role="tablist">`
Segmented	radio inputs inside `__segment`	`<label class="ts-segmented-control__segment"><input class="ts-segmented-control__input" type="radio">`
DataTable	`sort` → `aria-sort`; selected row → `data-selected`	`<table class="ts-data-table__table">`
TextField / Select / Textarea / SearchField	`size` → `data-size`; `state` → `data-state` (rest, readonly, disabled); `error` → `data-invalid` and `__error`	`<div class="ts-input__field" data-size="md" data-state="rest">`
Checkbox / Radio / Switch	`size` → `data-size`; `tone="danger"` on the switch	`<label class="ts-switch__checkable" data-size="md">`
Alert	`tone` → `data-tone`; `compact` → `__note` instead of `__banner`	`<div class="ts-alert ts-alert__banner" data-tone="warning">`
EmptyState	`tone="danger"` for load errors	`<div class="ts-empty-state">`
Avatar	`size` → `data-size`; `src` or initials	`<span class="ts-avatar" data-size="sm">`
Modal	`size` → `__panel[data-size]` (sm 400, md 520, lg 720, xl 1200)	`<div class="ts-modal__panel" data-size="md" role="dialog" aria-modal="true">`
Toast	`tone` → `data-tone`	`<div class="ts-toast" data-tone="success" role="status">`

5.2 Reference wrappers

```
// components/kit.tsx: the wrappers every page uses
import type { ButtonHTMLAttributes, InputHTMLAttributes, ReactNode } from 'react';

type Size = 'sm' | 'md' | 'lg';
type Tone = 'success' | 'info' | 'warning' | 'danger' | 'neutral' | 'violet';

export function Button({ variant = 'secondary', size = 'md', loading, children, ...p }:
  { variant?: 'primary' | 'accent' | 'secondary' | 'danger' | 'ghost'; size?: Size; loading?: boolean } &
ButtonHTMLAttributes<HTMLButtonElement>) {
  return (
    <button className="ts-button" data-variant={variant} data-size={size} data-state={loading ? 'loading' :
undefined} {...p}>
      <span className="ts-button__content"><span className="ts-button__label">{children}</span></span>
      {loading && <span className="ts-button__spinner"><span className="ts-loader"><span className="ts-loader__ring"
/></span></span>}
    </button>
  );
}

export const Badge = ({ tone = 'neutral', caps, solid, children }: { tone?: Tone; caps?: boolean; solid?: boolean;
children: ReactNode }) =>
  <span className="ts-badge" data-tone={tone} data-caps={caps || undefined} data-solid={solid || undefined}>
{children}</span>;

export function StatTile({ label, value, icon, tone = 'brand', change, caption }:
  { label: string; value: ReactNode; icon?: ReactNode; tone?: 'brand' | 'success' | 'danger' | 'warning' | 'info' |
'violet'; change?: { text: string; good?: boolean }; caption?: string }) {
  return (
    <article className="ts-stat-tile">
      <div className="ts-stat-tile__head">
        <div className="ts-stat-tile__lead"><p className="ts-stat-tile__label">{label}</p><p className="ts-stat-
tile__figure">{value}</p></div>
        {icon && <span className="ts-stat-tile__icon-chip" data-icon-tone={tone}>{icon}</span>}
      </div>
      {(change || caption) && (
        <p className="ts-stat-tile__change" data-tone={change ? (change.good ? 'positive' : 'negative') :
undefined}>
          {change && <span className="ts-stat-tile__change-value">{change.text}</span>}
          {caption && <span className="ts-stat-tile__change-caption">{caption}</span>}
        </p>
      )}
    </article>
  );
}

export function Panel({ title, subtitle, actions, children }: { title: string; subtitle?: string; actions?:
ReactNode; children: ReactNode }) {
  return (
    <section className="ts-panel">
      <div className="ts-panel__header">
        <div className="ts-panel__heading"><h2 className="ts-panel__title">{title}</h2>{subtitle && <p
className="ts-panel__subtitle">{subtitle}</p>}</div>
        {actions && <div className="ts-panel__actions">{actions}</div>}
      </div>
      <div className="ts-panel__body">{children}</div>
    </section>
  );
}

export function PageHeader({ crumbs = [], title, subtitle, actions }:
  { crumbs?: { label: string; href?: string }[]; title: string; subtitle?: string; actions?: ReactNode }) {
  return (
    <header className="ts-page-header">
      {crumbs.length > 0 && (
        <nav className="ts-page-header__breadcrumbs" aria-label="Breadcrumb"><ol className="ts-page-header__crumbs">
          {crumbs.map((c, i) => <li key={i} className="ts-page-header__crumb">
            <a className={`ts-page-header__crumb-link ${i === crumbs.length - 1 ? 'ts-page-header__current' : ''}`}
href={c.href} aria-current={i === crumbs.length - 1 ? 'page' : undefined}>{c.label}</a>
```

```
          </li>)}
        </ol></nav>
      )}
      <div className="ts-page-header__lead">
        <h1 className="ts-page-header__title">{title}</h1>
        {subtitle && <p className="ts-page-header__subtitle">{subtitle}</p>}
      </div>
      {actions && <div className="ts-page-header__actions">{actions}</div>}
    </header>
  );
}

export function TextField({ id, label, hint, error, size = 'md', ...p }:
  { id: string; label: string; hint?: string; error?: string; size?: Size } & InputHTMLAttributes<HTMLInputElement>)
{
  return (
    <div className="ts-input__field" data-size={size} data-state={p.disabled ? 'disabled' : p.readOnly ? 'readonly'
: 'rest'} data-invalid={error ? '' : undefined}>
      <label className="ts-input__label" htmlFor={id}>{label}</label>
      {hint && <p className="ts-input__hint">{hint}</p>}
      <div className="ts-input__control"><input className="ts-input" id={id} aria-invalid={!!error} aria-
describedby={error ? `${id}-err` : undefined} {...p} /></div>
      {error && <div className="ts-input__meta"><p className="ts-input__error" id={`${id}-err`}>{error}</p></div>}
    </div>
  );
}
```

Build the rest the same way, copying the markup of each component from `example.html` and its parts from the PDF's Part / Class table: tabs, segmented control, data table with its footer, select, search field, textarea, checkbox, radio, switch, alert, empty state, avatar, modal, toast, menu, popover, tooltip, filter bar, save bar, detail page, form page and link cards.

5.3 Behaviour the CSS does not give you

The kit is CSS only. Your wrappers must add the behaviour:

COMPONENT	BEHAVIOUR TO ADD
Sidebar groups	toggle `aria-expanded`; show or hide the children; open the group of the current page; remember the open groups
Drawer	toggle `data-drawer` on `.ts-app`; close on scrim click, Esc and navigation
Tabs	`role="tablist"` / `tab` / `tabpanel`, `aria-selected`, arrow keys between tabs
Modal	render the backdrop and panel only when open; trap focus inside; Esc and backdrop close it; return focus to the opener
Menu, popover, tooltip	place them (`position: fixed`; the kit reads `--ts-menu-top` / `--ts-menu-inline` and `--ts-popover-top` / `--ts-popover-inline`); close on outside click and Esc
Toast	a holder at the bottom centre, `z-index: var(--ts-z-toast)`; remove after about 4 seconds; `role="status"`
Data table	sorting (`aria-sort` on the header cell), selection (`data-selected`), pagination, an empty state in a notice cell
Checkbox indeterminate	set `input.indeterminate = true` in code; HTML can't
Colour mode	set or remove `data-ts-theme` on `<html>` and store the choice (2.4)

6. Check your result against the PDF

Do this for every screen before calling it done.

1. **Side by side.** Open the PDF page of each component next to your screen, at 100% zoom, light theme. The shapes, colours and spacing must match.
2. **Measure.** In the browser's developer tools, check the computed values of a few elements against the PDF's Property / Value tables:
   - the page title: Outfit, 38px, weight 600;
   - a stat-tile figure: Outfit, 32px;
   - a md button: 42px tall;
   - a nav row: 44px tall, radius 14;
   - a panel: radius 22, padding 20;
   - a table header cell: 11.5px, uppercase.
3. **Themes.** Switch to dark and compare with PDF chapter 10. No text may disappear. No white box may stay white.
4. **States** (PDF chapter 9):
   - tab through the page and see the green focus ring on every control;
   - hover rows and buttons;
   - disable a button (opacity .55);
   - make a field invalid (red border and error line).
5. **Sizes.** Check at 375px, 768px, 1280px and 1600px. The sidebar becomes a drawer below 64rem.
6. **Search for leftovers.** A screen is done when its code has no raw colour and no old class:

```
grep -rnE "#[0-9a-fA-F]{3,8}\b|rgb\(|bg-(blue|gray|slate|green|red)-[0-9]" app components --include=*.tsx --
include=*.css | grep -v design/tripssource-kit
```

7. Moving an existing product onto the kit

Don't rewrite everything at once. Do it in four steps, each one shippable. This is how Flowscope's studio was moved.

7.1 Step A: bridge the tokens (one day)

Point the product's existing colour and font names at the kit's tokens, in one file loaded after the kit. Every existing screen takes the new colours, fonts and dark mode at once, before a single component changes.

```
/* app/styles/kit-bridge.css: the old names, now driven by the kit */
:root, [data-ts-theme] {
  --color-paper: var(--ts-ground);
  --color-paper-raised: var(--ts-surface);
  --color-ink: var(--ts-text);
  --color-ink-muted: var(--ts-text-secondary);
  --color-hairline: var(--ts-border);
  --color-brand: var(--ts-brand);
  --color-navy: var(--ts-brand-deep);
  --color-green: var(--ts-success-solid);
  --color-amber: var(--ts-warning-solid);
  --color-oxblood: var(--ts-danger-solid);
  --font-sans: var(--ts-font-body);
  --font-display: var(--ts-font-display);
}
```

Check the selectors: the bridge must win over the product's own light and dark blocks. Use the same selectors they use, or one step more specific, and load it after them.

7.2 Step B: the frame (one to two days)

Replace the app shell (sidebar, top bar, layout) with the kit's frame from step 3. Keep the product's menu data; only the rendering changes. If the product has its own base styles (a CSS reset, Tailwind's `preflight`), keep them and leave out `data-ts-reset`. Then copy only the kit's typography rules you need into the product's CSS file (2.6).

7.3 Step C: the shared components (two to four days)

Rewrite the product's shared components (Button, Card, Badge, Table, PageHead, Tabs…) as the wrappers in 5.2, keeping their names and props. Every page that uses them changes at once. Do the most-used first.

7.4 Step D: page by page

For each page: choose its pattern (step 4), replace hand-made boxes with kit components, run the check in step 6, and ship. Delete the old CSS a component used once no page uses it.

8. Notes for the DCLab product (`decision_ai/apps/web`)

What I found in the code (read only, nothing changed):

- **Stack:** Next.js 15 (App Router), React 18, Tailwind 3.4, `lucide-react` icons.
- **Fonts:** bundled with `next/font/local` under `app/fonts/` (Inter, Manrope, JetBrains Mono). The browser never calls Google, which is founder decision D1.
- **Tokens:**
  - marketing pages: `--color-*` in `app/globals.css`;
  - the signed-in Studio: its own design system in `components/studio/studio.css`, scoped under `.studio` (teal accent, violet for AI, Manrope headings), also from D1.
- **Studio components:** in `components/studio/`: AgentText, Banner, Card, Chart, Chat, CodeBlock, ComingNext, CommandBar, Crumbs, DataTable, EventList, FlowBar, GlossaryDialog, GlossaryTerm, KeyValue, Level, PageGuide, PageHead, Pill, ReviewNotes, ScreenStatusTag, SectionTabs, Shell, Stat, StepBar, Tabs, Term.
- **Shell:** `app/components/layout/`: AppShell, AppSidebar, AppMobileDrawer, CommandPalette, WorkspaceSelector.

Before you start: the TripsSource look replaces the look decided in D1. Record the change as a new founder decision in that repository (the owner's call), and say in it that the fonts stay bundled locally.

Plan:

1. **Fonts.** Add Outfit and Plus Jakarta Sans as variable woff2 files under `app/fonts/` with their `OFL.txt`. Load them in `app/layout.tsx` with `next/font/local`, as `--ts-font-display` and `--ts-font-body`. Keep JetBrains Mono for code (`--ts-font-mono`).
2. **Kit.** Copy the kit to `apps/web/design/tripssource-kit/`. Import the three CSS files in `app/layout.tsx` before `globals.css`.
3. **Bridge.** Point the `.studio` tokens at the kit, inside `.studio`, so marketing pages are untouched:

```
.studio, .studio[data-theme] {
  --bg: var(--ts-ground); --surface: var(--ts-surface); --surface-2: var(--ts-surface-sunken); --surface-3: var(-
-ts-surface-muted);
  --ink: var(--ts-text); --ink-2: var(--ts-text-secondary); --muted: var(--ts-text-muted);
  --line: var(--ts-border); --line-strong: var(--ts-border-strong);
  --accent: var(--ts-brand); --accent-strong: var(--ts-brand-deep); --accent-ink: var(--ts-on-brand); --accent-
soft: color-mix(in oklab, var(--ts-brand) 11%, var(--ts-surface));
  --accent-text: var(--ts-brand-deep);
  --ok: var(--ts-success-solid); --warn: var(--ts-warning-solid); --crit: var(--ts-danger-solid);
  --ok-text: var(--ts-success-fg); --warn-text: var(--ts-warning-fg); --crit-text: var(--ts-danger-fg);
  --primary: var(--ts-brand-deep); --primary-ink: var(--ts-on-brand);
  --font-display: var(--ts-font-display); --font-body: var(--ts-font-body);
  --r: var(--ts-radius-card); --r-sm: var(--ts-radius-sm); --nav-w: var(--ts-sidebar-width);
}
```

   - Keep `--ai` and `--ai-soft` (violet for AI). The kit has the same colours as `--ts-violet-fg` and `--ts-violet-chip`, so AI stays recognisable.
   - The Studio switches dark mode with `data-theme` on `.studio`; the kit uses `data-ts-theme`. Set both from the one colour-mode control.
4. **Shell.** Rebuild `AppShell` and `AppSidebar` (and `components/studio/Shell.tsx`) on the kit's frame (step 3).
   - Keep `app-navigation.ts` and `sidebar-config.ts` as the menu data.
   - The workspace selector becomes a kit select in the sidebar's first section.
   - The command palette opens from `.ts-topbar__search`.
5. **Components.** Rebuild each Studio component on its kit counterpart, keeping its name and props:

STUDIO COMPONENT	KIT COMPONENT
Card	Panel (`.ts-panel`), or View card for detail pages

STUDIO COMPONENT	KIT COMPONENT
Stat	Stat tile (`.ts-stat-tile`)
Pill, ScreenStatusTag, Level	Badge (`.ts-badge` , tone and caps)
Banner, PageGuide	Alert (`.ts-alert__banner`); a compact one as `.ts-alert__note`
PageHead, Crumbs	Page header (`.ts-page-header` , breadcrumbs)
Tabs, SectionTabs	Tabs (underline for sections, pill for views)
DataTable	Data table + table footer
KeyValue	Detail page facts and list (`.ts-detail-page__facts` , `__list`)
StepBar, FlowBar	Segmented control for choices; a sequence of badges or the kit's progress for steps
EventList, ReviewNotes	View rows (`.ts-view__rows`) inside a panel
GlossaryTerm, Term	Info term (`.ts-info-term__term`) with a tooltip or popover
GlossaryDialog	Modal (md)
CommandBar	Filter bar (`.ts-filter-bar`)
ComingNext	Empty state
CodeBlock, Chat, AgentText, Chart	Keep; restyle their frames with Panel and their colours with `--ts-*` (charts use `--ts-chart-1…8`)

6. **Verify** with the repository's own commands after each step, plus the check in step 6:

```
npx tsc --noEmit && npm run lint && npm run test:components
```

   Add the e2e suite (`npm run e2e`) when the shell changes.

9. Notes for DCLab R&D

DCLab R&D's own pages are static HTML: `docs/product-demo/` , `docs/recap/` , `docs/copilot_demo.html` , the notebook-companion views. For each:

1. Copy the kit next to them, or link to one shared copy.
2. Add the three `<link>` tags and the fonts (2.2, 2.3) to the `<head>` .
3. Wrap the page in the frame (step 3) if it has navigation, or in a canvas if it is a single document.
4. Replace hand-made boxes with kit components, using `example.html` as the source of markup.

Flowscope only reads DCLab R&D and never writes there. These changes are made in the R&D repository by its owner.

10. A prompt for a coding agent

To have an agent (Claude Code, Codex, Cursor) do the work in the target repository, copy the kit folder there first, then give it this:

Implement the TripsSource design kit in this repository.

Source of truth: `design/tripssource-kit/` :

- `TripsSource-Design-Kit.pdf` (the target look and every measurement);
- `ts-tokens.css` , `ts-theme-blue.css` , `ts-components.css` (the implementation; load them, never retype their values);
- `example.html` (working markup for every component);
- `IMPLEMENTING.md` (the steps).

Steps: follow IMPLEMENTING.md in order.

1. Install the fonts locally (section 2.2, bundled; no request to Google) and load the three CSS files (2.3).
2. Add the token bridge (7.1, or section 8 for this repository's `.studio` tokens).
3. Rebuild the app frame (step 3), keeping the existing menu data.
4. Rewrite the shared components as kit wrappers (5.2), keeping their names and props.
5. Move pages one by one onto the five patterns (step 4).

Rules:

- Only `var(--ts-*)` tokens in new CSS; no hex values, no raw Tailwind colours.
- Own classes get a product prefix, never `.ts-` .
- Keep every existing behaviour, route and test.
- Light, dark and system must all work.

After each step:

- run the repository's type-check, lint and tests;
- compare the screens with the PDF at 375, 768, 1280 and 1600px wide, light and dark (section 6);
- commit, with the step in the message.

Stop and ask before changing anything the repository records as a decision (for example its fonts or its colours).

11. Mistakes to avoid

MISTAKE	INSTEAD
Retyping colours or sizes from the PDF into new CSS	Use the token, `var(--ts-…)`
Editing the kit's files to change one product detail	Override the token in your own file loaded after the kit (2.6)
Building a card, badge or button by hand "just here"	Use the wrapper; add a variant to it if one is missing
Forgetting the dark blocks when overriding a colour	Override in `:root, [data-ts-theme]` and in both dark blocks
Outfit for body text, or caps labels longer than three words	Outfit only for titles and numbers; caps labels stay short
More than one primary button on a screen	One primary; the rest secondary or ghost
The accent colour for buttons or large areas	The accent is a marker only (focus, current, underline); actions use brand-deep
Hiding focus outlines	Keep the kit's focus ring on every control
Loading fonts from Google in an app that must not	Bundle them with `next/font/local`
A blank screen while loading or when empty	A skeleton or ring, then an empty state

12. Keeping the kit in step

- The kit lives in Flowscope at `design/tripssource-kit/`. Update it there, then copy the folder to the products.
- After changing a token or a component, rebuild the PDF so it still matches:

```
python3 design/tripssource-kit/pdf/build.py
```

- Add a line in the product's changelog naming the kit version (the Flowscope commit) it was copied from.